

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №1

Тема: Визначення алгоритму. Способи описання та властивості, класи алгоритмів

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати поняття алгоритму та його основні властивості, ознайомитися з різними способами опису алгоритмів (словесним, графічним, псевдокодом, мовою програмування), навчитися розрізняти базові керуючі структури алгоритмів (лінійну, розгалуження, циклічну), а також класифікувати алгоритми за різними ознаками (детерміновані/імовірнісні, точні/наближені).

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), олівець і папір (або графічний редактор) для побудови блок-схем.

Короткі теоретичні відомості

1. Означення алгоритму. Алгоритм — це скінченна впорядкована послідовність точно визначених дій (приписів), виконання яких дозволяє розв'язати задачу певного класу за скінченну кількість кроків, перетворюючи вхідні дані на вихідний результат.

2. Властивості алгоритму:

- **Дискретність** — алгоритм складається з окремих, чітко відокремлених один від одного елементарних кроків (приписів), що виконуються послідовно, один за одним, а не безперервно.
- **Визначеність (детермінованість)** — кожен крок алгоритму повинен бути однозначно й точно визначеним, не допускати довільного (суб'єктивного) тлумачення виконавцем; за одних і тих самих вхідних даних детермінований алгоритм завжди дає той самий результат і виконує ту саму послідовність дій.
- **Результативність (скінченність)** — алгоритм повинен завершувати роботу за скінченну кількість кроків і видавати конкретний результат (або повідомлення про неможливість розв'язання).
- **Масовість (загальність)** — алгоритм застосовний не до одного конкретного набору вхідних даних, а до цілого класу задач, тобто до будь-яких вхідних даних, що задовольняють певним, наперед визначеним умовам.
- **Ефективність (коректність)** — кожен окремий крок алгоритму має бути реально виконуваним виконавцем за скінченний час наявними засобами, а підсумковий результат — правильним розв'язком поставленої задачі.

3. Способи опису алгоритмів:

- **Словесний (природномовний) опис** — алгоритм описується звичайною мовою у вигляді пронумерованих кроків-приписів. Переваги: зрозумілість без спеціальної підготовки. Недоліки: можлива неоднозначність формулювань, громіздкість, складність формалізації для подальшого програмування.
- **Формульно-словесний (аналітичний) опис** — алгоритм подається через математичні формули та вирази з текстовими поясненнями порядку їх застосування.
- **Графічний опис (блок-схема, flowchart)** — алгоритм зображується у вигляді послідовності стандартизованих геометричних фігур (блоків), кожна з яких позначає певний тип дії, з'єднаних лініями (стрілками), що показують порядок виконання. Основні типи блоків: овал — початок/кінець алгоритму; паралелограм — введення/виведення даних; прямокутник — обробка даних (виконання дій, обчислення); ромб — перевірка умови (розгалуження). Переваги: наочність, легкість простежування логіки виконання.
- **Псевдокод** — проміжна форма запису між природною мовою та мовою програмування; використовує ключові слова (якщо...то...інакше, поки, для, почати, кінець тощо) без суворого дотримання синтаксису конкретної мови програмування, що робить його зручним для проектування алгоритму до вибору мови реалізації.
- **Структурограми (діаграми Нассі–Шнейдермана)** — графічний спосіб опису без стрілок, у вигляді системи вкладених прямокутних блоків, що явно відображає структуру вкладеності керуючих конструкцій.
- **Мова програмування** — запис алгоритму у вигляді програмного коду конкретною мовою програмування; найбільш формалізований і однозначний спосіб опису, безпосередньо придатний для виконання комп'ютером.

4. Базові керуючі структури алгоритмів. Згідно з теоремою Бьома–Якопіні, будь-який алгоритм (обчислюваний за Тьюрингом) можна побудувати, використовуючи лише три базові керуючі структури:

- **Лінійна структура (слідування)** — дії виконуються одна за одною, строго в тому порядку, в якому вони записані в алгоритмі, без розгалужень і повторень.
- **Розгалуження (умовна структура)** — залежно від виконання певної логічної умови обирається один з кількох (найчастіше — двох) можливих напрямків подальшого виконання алгоритму (конструкції "якщо...то...інакше", "вибір за варіантами").
- **Циклічна структура (повторення)** — група дій виконується багаторазово, доки виконується (цикл з передумовою чи післяумовою) або не виконується задана умова, або задану кількість разів (цикл з параметром).

5. Класифікація алгоритмів за іншими ознаками:

- **За характером виконання: детерміновані** алгоритми — за тих самих вхідних даних завжди дають той самий результат і виконують ту саму послідовність дій; **імовірнісні (стохастичні, рандомізовані)** алгоритми — використовують у своїй роботі випадкові числа, тому результат (або сам хід виконання) може відрізнитися від запуску до запуску навіть за однакових вхідних даних (наприклад, методи Монте-Карло, рандомізований QuickSort).
- **За точністю результату: точні** алгоритми — гарантовано дають абсолютно точний розв'язок задачі; **наближені (евристичні)** алгоритми — дають розв'язок із заданою точністю або "достатньо хороший" розв'язок за прийнятний час, застосовуються, коли отримання точного розв'язку обчислювально неможливе або невиправдано дороге (наприклад, чисельні методи розв'язування нелінійних рівнянь, евристики для NP-складних задач).
- **За способом організації обчислень: послідовні** алгоритми — дії виконуються одна за одною на одному обчислювальному пристрої; **паралельні** алгоритми — кілька дій можуть виконуватися одночасно на різних процесорах або ядрах.

- **За рекурсивністю:** рекурсивні алгоритми — визначаються через виклик самих себе для простіших підзадач; **ітеративні** алгоритми — реалізують повторення через явні цикли, без самовиклику.

Хід роботи

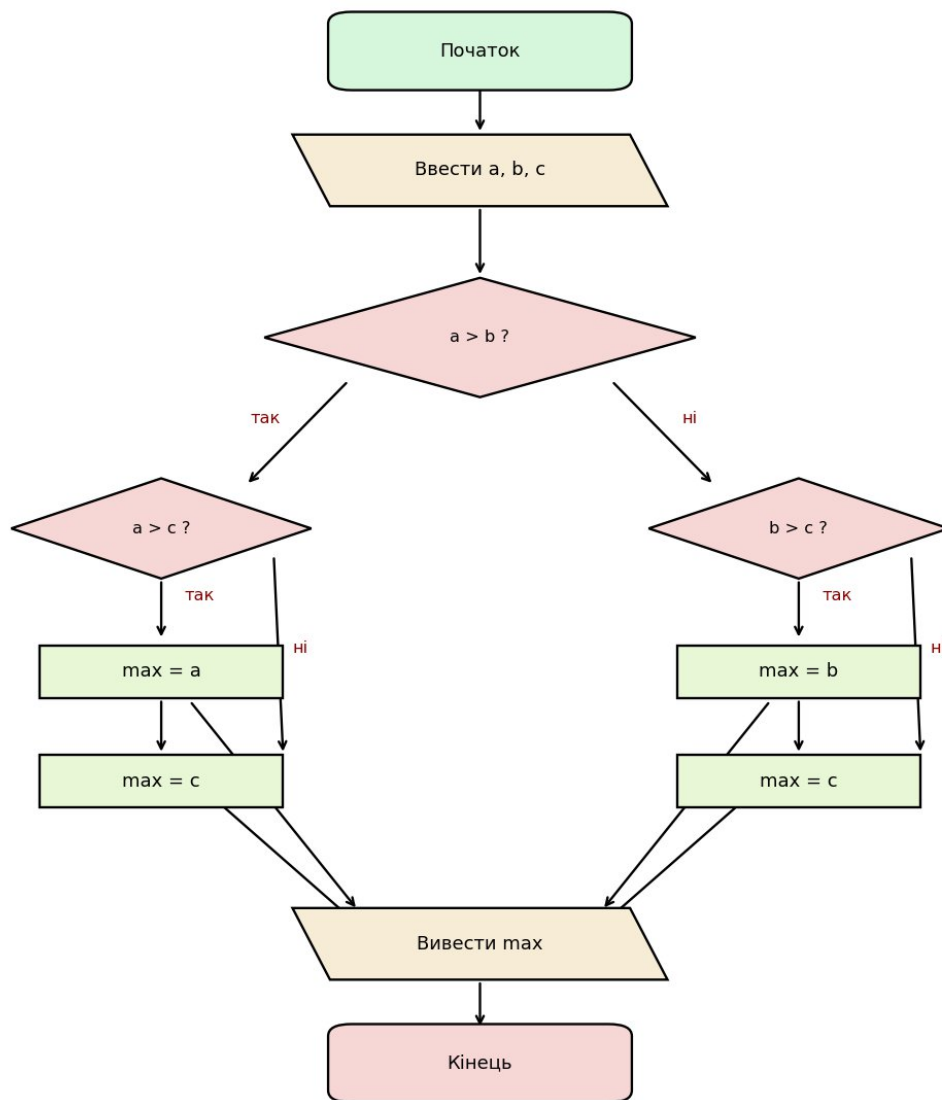
Завдання 1. Опис одного алгоритму трьома різними способами

Для задачі "визначити найбільше з трьох заданих чисел a , b , c " скласти опис алгоритму трьома способами: словесним, графічним (блок-схема) та у вигляді псевдокоду.

Порядок виконання:

1. Складіть **словесний опис**: сформулюйте алгоритм у вигляді пронумерованих кроків природною мовою (наприклад: "Крок 1. Ввести три числа a , b , c . Крок 2. Порівняти a і b . Якщо a більше за b , перейти до кроку 3, інакше — до кроку 5. ..." і так далі до виведення результату).
2. Побудуйте **блок-схему**: намалюйте на папері (або в графічному редакторі) послідовність стандартних блоків — овал "Початок", паралелограм "Ввести a , b , c ", ромби для перевірки умов " $a > b$?" та подальших " $a > c$?" / " $b > c$?", прямокутники для присвоєння $max=a$, $max=b$ або $max=c$, паралелограм "Вивести max ", овал "Кінець" — з'єднавши їх стрілками відповідно до логіки алгоритму (звертеся зі зразком блок-схеми, наведеним нижче в результатах виконання).
3. Складіть **псевдокод**: запишіть той самий алгоритм, використовуючи умовні ключові слова (ЯКЩО...ТО...ІНАКШЕ), без прив'язки до синтаксису конкретної мови програмування.
4. Порівняйте всі три отримані описи одного й того самого алгоритму: визначте, який з них найзручніший для розуміння людиною без технічної підготовки, а який — найзручніший як проміжний крок перед написанням програмного коду.

Приклад блок-схеми для цього алгоритму:



Приклад псевдокоду:

```

ПОЧАТОК
  ввести a, b, c
  ЯКЩО a > b ТО
    ЯКЩО a > c ТО
      max ← a
    ІНАКШЕ
      max ← c
    КІНЕЦЬ ЯКЩО
  ІНАКШЕ
    ЯКЩО b > c ТО
      max ← b
    ІНАКШЕ
      max ← c
    КІНЕЦЬ ЯКЩО
  КІНЕЦЬ ЯКЩО
  вивести max
КІНЕЦЬ
  
```

Завдання 2. Аналіз властивостей алгоритмів

Дано три текстові описи "алгоритмів". Для кожного визначити, які з властивостей алгоритму (дискретність, визначеність, скінченність, масовість, ефективність) виконуються, а які порушуються, з обґрунтуванням.

Описи для аналізу:

Опис А: "Візьми число. Ділити його на 2, поки не набридне."

Опис Б (алгоритм Евкліда для НСД двох натуральних чисел): "Крок 1. Якщо $b = 0$, то відповідь — a , завершити роботу. Крок 2. Інакше обчислити остачу r від ділення a на b . Крок 3. Присвоїти $a := b$, $b := r$. Крок 4. Повернутися до кроку 1."

Опис В (кулінарний рецепт): "Візьми борошно. Додай трохи солі за смаком. Виміси тісто до потрібної консистенції. Випікай, доки не підрум'яниться."

Порядок виконання:

1. Для опису А визначте, яка властивість порушена найочевидніше (зверніть увагу на формулювання умови зупинки "поки не набридне") і поясніть, чому ця умова не є коректним критерієм завершення алгоритму (порушення скінченності/результативності), а також чи можна однозначно визначити результат (визначеність).
2. Для опису Б перевірте виконання кожної з п'яти властивостей по черзі: чи розбитий алгоритм на чіткі кроки (дискретність); чи кожен крок однозначний (визначеність); чи гарантовано алгоритм завершиться за скінченну кількість кроків для будь-яких натуральних a , b (скінченність — можна обґрунтувати тим, що остача r завжди менша за b , тому послідовність значень b строго спадає); чи застосовний алгоритм до цілого класу задач, а не лише до конкретних чисел (масовість); чи кожен крок реально виконуваний (ефективність).
3. Для опису В визначте, які формулювання є неоднозначними (наприклад, "трохи солі за смаком", "до потрібної консистенції", "доки не підрум'яниться") і яку саме властивість алгоритму вони порушують (визначеність — оскільки різні виконавці витлумачать ці вказівки по-різному й отримають різний результат).
4. Складіть підсумкову таблицю "Опис — Дискретність — Визначеність — Скінченність — Масовість — Ефективність" із позначками "виконується"/"порушується" для кожного з трьох описів та коротким обґрунтуванням.

Завдання 3. Програмна реалізація трьох базових керуючих структур

Реалізувати мовою Python по одному прикладу алгоритму для кожної з трьох базових керуючих структур: лінійної, розгалуження та циклічної, перевірити коректність кожної реалізації.

Порядок виконання:

1. Для **лінійної структури** реалізуйте функцію обчислення площі трикутника за формулою Герона: спочатку обчисліть півпериметр $p = (a+b+c)/2$, потім площу $area = \sqrt{p * (p-a) * (p-b) * (p-c)}$ — зверніть увагу, що дії виконуються строго послідовно, без жодних розгалужень чи повторень.
2. Для **розгалуження** реалізуйте функцію `max_of_three(a, b, c)` за алгоритмом із завдання 1, використовуючи вкладені конструкції `if...else`.
3. Для **циклічної структури** реалізуйте функцію обчислення суми цифр натурального числа: у циклі `while n > 0` виокремлюйте останню цифру операцією `n % 10`,

додавайте її до суми, і "відкидайте" останню цифру числа операцією цілочисельного ділення $n // = 10$.

4. Перевірте кожну функцію на кількох тестових наборах даних, звіряючи результат з еталонним (наприклад, вбудованою функцією `max()` для другого прикладу, або `sum(int(d) for d in str(n))` для третього).
5. Проаналізуйте код кожної з трьох функцій і визначте, яка керуюча структура (лінійна, розгалуження, циклічна) у ній переважає.

Приклад реалізації мовою Python:

```
python
import math

# 1. ЛІНІЙНИЙ алгоритм — площа трикутника за формулою Герона
def triangle_area(a, b, c):
    p = (a + b + c) / 2
    area = math.sqrt(p * (p-a) * (p-b) * (p-c))
    return area

print("площа трикутника (3,4,5) =", triangle_area(3, 4, 5))

# 2. РОЗГАЛУЖЕННЯ — максимум із трьох чисел
def max_of_three(a, b, c):
    if a > b:
        if a > c:
            return a
        else:
            return c
    else:
        if b > c:
            return b
        else:
            return c

for a, b, c in [(3,7,5), (10,2,8), (1,1,1), (-5,-2,-8)]:
    result = max_of_three(a, b, c)
    assert result == max(a, b, c)
    print(f"max({a},{b},{c}) = {result}")

# 3. ЦИКЛІЧНИЙ алгоритм — сума цифр числа
def digit_sum(n):
    total = 0
    while n > 0:
        digit = n % 10
        total += digit
        n //= 10
    return total

for n in [12345, 999, 7, 1080]:
    result = digit_sum(n)
    assert result == sum(int(d) for d in str(n))
    print(f"сума цифр({n}) = {result}")
```

Результат виконання програми:

```
площа трикутника (3,4,5) = 6.0
max(3,7,5) = 7
max(10,2,8) = 10
max(1,1,1) = 1
max(-5,-2,-8) = -2
сума цифр(12345) = 15
сума цифр(999) = 27
```

сума цифр(7) = 7
сума цифр(1080) = 9

Усі три реалізації підтверджені збігом з еталонними обчисленнями.

Завдання 4. Порівняння детермінованого й імовірнісного класів алгоритмів

Реалізувати два алгоритми обчислення числа π : детермінований (ряд Лейбніца) та імовірнісний (метод Монте-Карло), продемонструвати на практиці ключову відмінність цих класів — детермінований алгоритм завжди дає той самий результат за тих самих вхідних даних, тоді як імовірнісний — дає різний результат від запуску до запуску.

Порядок виконання:

1. Реалізуйте функцію `pi_leibniz(n_terms)`, що обчислює наближення числа π за формулою ряду Лейбніца: $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$, підсумовуючи `n_terms` доданків у циклі з почерговою зміною знаку.
2. Реалізуйте функцію `pi_monte_carlo(n_points)`, що методом Монте-Карло оцінює число π : у циклі `n_points` разів генеруйте випадкову точку (x, y) у квадраті $[-1, 1] \times [-1, 1]$ за допомогою `random.uniform(-1, 1)`; підраховуйте, скільки точок потрапило всередину одиничного кола (умова $x^2 + y^2 \leq 1$); оцінка π обчислюється як $4 * (\text{кількість точок у колі}) / (\text{загальна кількість точок})$ (відношення площі кола до площі квадрата дорівнює $\pi/4$).
3. Запустіть `pi_leibniz` кілька разів (наприклад, 3 рази) з однаковим значенням `n_terms` і переконайтесь, що результат абсолютно однаковий щоразу — це і є прояв детермінованості.
4. Запустіть `pi_monte_carlo` кілька разів (наприклад, 5 разів) з однаковим значенням `n_points` і зафіксуйте, що результат щоразу трохи відрізняється — це і є прояв імовірнісної (стохастичної) природи алгоритму.
5. Дослідіть збіжність обох алгоритмів: для зростаючих значень параметра (`n_terms` або `n_points` відповідно: 10, 100, 1000, 10000, 100000, 1000000) обчисліть похибку відносно точного значення `math.pi` і простежте, як вона зменшується.
6. Порівняйте швидкість збіжності: який з двох алгоритмів досягає меншої похибки при однаковій кількості обчислювальних "кроків" (доданків ряду проти згенерованих точок).

Приклад реалізації мовою Python:

```
python
import random, math

# ДЕТЕРМІНОВАНИЙ алгоритм: ряд Лейбніца
def pi_leibniz(n_terms):
    total = 0.0
    sign = 1
    for k in range(n_terms):
        total += sign / (2*k + 1)
        sign *= -1
    return 4 * total

# ІМОВІРНІСНИЙ алгоритм: метод Монте-Карло
def pi_monte_carlo(n_points):
    inside = 0
    for _ in range(n_points):
        x, y = random.uniform(-1, 1), random.uniform(-1, 1)
```

```

    if x*x + y*y <= 1:
        inside += 1
    return 4 * inside / n_points

print("Детермінований алгоритм — повторні запуски з тими самими n_terms:")
for _ in range(3):
    print(f" pi_leibniz(100000) = {pi_leibniz(100000):.6f}")

print("\nІмовірнісний алгоритм — повторні запуски з тими самими n_points:")
for _ in range(5):
    print(f" pi_monte_carlo(100000) = {pi_monte_carlo(100000):.6f}")

print(f"\nІстинне значення pi = {math.pi:.6f}")

```

Результат виконання програми:

Детермінований алгоритм — повторні запуски з тими самими n_terms:

```

pi_leibniz(100000) = 3.141583
pi_leibniz(100000) = 3.141583
pi_leibniz(100000) = 3.141583

```

Імовірнісний алгоритм — повторні запуски з тими самими n_points:

```

pi_monte_carlo(100000) = 3.136520
pi_monte_carlo(100000) = 3.150920
pi_monte_carlo(100000) = 3.138720
pi_monte_carlo(100000) = 3.139360
pi_monte_carlo(100000) = 3.136040

```

Істинне значення pi = 3.141593

Дослідження збіжності:

Ряд Лейбніца (детермінований):

```

n=   10: похибка = 9.98e-02
n=  100: похибка = 1.00e-02
n= 1000: похибка = 1.00e-03
n= 10000: похибка = 1.00e-04
n= 100000: похибка = 1.00e-05
n= 1000000: похибка = 1.00e-06

```

Монте-Карло (імовірнісний):

```

n=   10: похибка = 8.58e-01
n=  100: похибка = 3.38e-01
n= 1000: похибка = 5.44e-02
n= 10000: похибка = 4.79e-03
n= 100000: похибка = 9.27e-04
n= 1000000: похибка = 1.07e-03

```

Висновок за результатами: детермінований алгоритм (ряд Лейбніца) при повторних запусках з однаковими параметрами видає **абсолютно ідентичний** результат щоразу (3.141583 у всіх трьох запусках) — це пряме підтвердження властивості визначеності.

Імовірнісний алгоритм (Монте-Карло) при тих самих параметрах видає **різний** результат кожного разу (від 3.136040 до 3.150920) через залежність від генератора випадкових чисел – це ілюструє принципову відмінність класу стохастичних алгоритмів. Щодо швидкості збіжності: ряд Лейбніца демонструє чітку закономірність – похибка зменшується приблизно пропорційно $1/n$ (кожне збільшення n у 10 разів зменшує похибку приблизно в 10 разів), тоді як метод Монте-Карло збігається значно повільніше й похибка поводить себе нерегулярно (не завжди монотонно зменшується) – це типова риса ймовірнісних алгоритмів, похибка яких зменшується лише "в середньому" й пропорційно $1/\sqrt{n}$, а не за суворо детермінованим законом.

Контрольні питання

1. Сформулюйте означення алгоритму.
2. Перелічіть основні властивості алгоритму та коротко поясніть кожну з них.
3. Яка властивість порушується у формулюванні "виконуй дію, поки не набридне"? Обґрунтуйте.
4. Назвіть основні способи опису алгоритмів. У чому переваги й недоліки кожного з них?
5. Які стандартні геометричні фігури (блоки) використовуються в блок-схемах і що кожна з них позначає?
6. Сформулюйте теорему Бьома–Якопіні. Які три базові керуючі структури вона визначає?
7. Чим детермінований алгоритм відрізняється від імовірнісного (стохастичного)? Наведіть приклад кожного.
8. Чим точний алгоритм відрізняється від наближеного (евристичного)? У яких випадках виправдане застосування наближених алгоритмів?
9. Наведіть приклад задачі, яку можна розв'язати і рекурсивним, і ітеративним алгоритмом.

Зміст звіту

1. Тема, мета роботи.
2. Три способи опису алгоритму (словесний, блок-схема, псевдокод) — завдання 1.
3. Таблиця аналізу властивостей для трьох описів — завдання 2.
4. Тексти програм та результати їх роботи (завдання 3, 4).
5. Висновки.